

Using Container Structure Tests for Container Acceptance Testing

Carrying out container acceptance testing early saves precious time, money and the cloud service provider's reputation, and avoids hard-to-fix infra downtimes. Container Structure Tests have been designed for this purpose.



Misconfiguration has been a major problem in the IT world for a long time. It gives birth to a disconnect between software versions, configs, processes, etc, while taking software from development to production. Who can forget the painful meetings wondering where things went wrong, and doing some patchwork to make the software work? In fact, software misconfiguration was found to be the root cause for many recent public cloud outages even for tech giants like Amazon, Google and Microsoft. So how do we address this issue?

Containers have largely replaced running native applications directly on modern cloud infrastructure. But these present a few challenges at runtime. The container images are expected to have a minimal footprint to launch and run containerised applications dynamically and properly. Acceptance testing becomes very important in the case of containers, as software version and configuration disconnects can be a headache. Unfortunately, I have witnessed even industry folks creating containers using images without any standards and then suffering major outages due to their unstable lifecycle.

The good news is that we have some great free and open source tools that can enhance the quality of code for our container workloads. We'll explore these tools for acceptance testing of containers that run independently in Docker Compose stack and Kubernetes mechanisms, respectively. I have tested everything on my Ubuntu 18.04 LTS on an AMD64 laptop and the latest Raspberry PI OS on RPI4, respectively. But the code examples should run on any modern GNU/Linux based machine able to run on a Docker engine.

Structural acceptance testing for containers with CST

How do you verify a server configured to run a single service or a set of services? You get into the server and run a set of commands to verify the required packages are installed with proper versions; you check their configuration files have correct values, the required services are configured to start automatically, those services are running and listening on the configured ports, etc. You can add tests for acceptance testing to cover more possibilities of misconfigurations, e.g., the required disk space, partitions and their file systems, required amount of RAM, leftover large artifacts not required, etc. So, you are asking your server a set of questions and expecting the correct answers as per the specifications required to run a correctly configured server. The modern automated way to do this is that you bake all the necessary acceptance tests in your server, and the server gives you a report on demand about which tests have been passed or failed.

The lifecycle of Docker containers starts with creating an image that packs in all the necessary artifacts. So our first acceptance testing tool is based on the structure of the image, verifying all the required (and must be absent) artifacts and the properties of the artifacts packed in. You can also call it static testing, as it's the image build time verification rather than the runtime verification. Our tool of choice, the Container Structure Tests (CST), is all about verifying container images to ensure that these consist of only working artifacts; this will save us from the runtime blues. Let's quickly see this in action by creating a Docker image with the command:

```
docker build --no-cache --progress plain . -f Dockerfile_
CSTestPrblm -t ctestprblm:1.19.3
```

You will need to create the dockerfile shown below before running the command:

```
--- Start of Dockerfile_CSTestPrblm ---

ARG ALPINEVER=3.20
```